

CHAPTER 16

POSTDELIVERY MAINTENANCE



16.1 Development and Maintenance

- Postdelivery maintenance
 - *Any* change to *any* component of the product (including documentation) after it has passed the acceptance test

16.2 Why Postdelivery Maintenance Is Necessary?

- Corrective maintenance
 - To correct residual faults
 - Analysis, design, implementation, documentation, or any other type of faults

16.2 Why Postdelivery Maint. Is Necessary (contd)?

- Perfective maintenance
 - Client requests changes to improve product effectiveness
 - Add additional functionality
 - Make product run faster
 - Improve maintainability

16.2 Why Postdelivery Maint. Is Necessary (contd)?

- Adaptive maintenance
 - Responses to changes in the environment in which the product operates
 - The product is ported to a new compiler, operating system, and/or hardware
 - A change to the tax code
 - 9-digit ZIP codes

16.3 What Is Required of Postdelivery Maintenance Programmers?

- At least 67% of the total cost of a product accrues during postdelivery maintenance
- Nevertheless, even today many organizations assign maintenance to
 - Unsupervised beginners, and
 - Less competent programmers

16.3 What is Required of Postd. Maint. Prog. (contd)?

- Postdelivery maintenance is one of the most difficult aspects of software production because
 - Postdelivery maintenance incorporates aspects of all other workflows

16.3 What is Required of Postd. Maint. Prog. (contd)?

- Suppose a defect report is handed to a maintenance programmer
 - Recall that a “defect” is a generic term for a fault, failure, or error
- What is the cause?
 - Nothing may be wrong
 - The user manual may be wrong, not the code
 - Usually, however, there is a fault in the code

16.3 What is Required of Postd. Maint. Prog. (contd)?

- Corrective Maintenance?
 - What tools does the maintenance programmer have to find the fault?
 - The defect report filed by user
 - The source code
 - And often nothing else

16.3 What is Required of Postd. Maint. Prog. (contd)?

- Corrective Maintenance (contd)?
 - A maintenance programmer must therefore have superb debugging skills
 - The fault could lie anywhere within the product
 - The original cause of the fault might lie in the by now nonexistent specifications or design artifacts

16.3 What is Required of Postd. Maint. Prog. (contd)?

- Corrective Maintenance (contd)?
 - Suppose that the maintenance programmer has located the fault
 - Problem:
 - How to fix it without introducing a regression fault

16.3 What is Required of Postd. Maint. Prog. (contd)?

- Corrective Maintenance (contd)?
 - How to minimize regression faults
 - Consult the detailed documentation for the product as a whole
 - Consult the detailed documentation for each individual module
 - What usually happens
 - There is no documentation at all, or
 - The documentation is incomplete, or
 - The documentation is faulty

16.3 What is Required of Postd. Maint. Prog. (contd)?

- Corrective Maintenance (contd)?
 - The programmer must deduce from the source code itself all the information needed to avoid introducing a regression fault
 - The programmer now changes the source code

16.3 What is Required of Postd. Maint. Prog. (contd)?

- Corrective Maintenance (contd)?
 - The programmer now must:
 - Test that the modification works correctly
 - Using specially constructed test cases
 - Check for regression faults
 - Using stored test data
 - Add the specially constructed test cases to the stored test data for future regression testing
 - Document all changes

16.3 What is Required of Postd. Maint. Prog. (contd)?

- Corrective Maintenance (contd)?
 - Major skills are required for corrective maintenance
 - Superb diagnostic skills
 - Superb testing skills
 - Superb documentation skills



16.3 What is Required of Postd. Maint. Prog. (contd)?

- Adaptive and Perfective Maintenance?
 - The maintenance programmer must go through the requirements, specifications, design and implementation workflows, using the existing product as a starting point
 - For some changes, additional code artifacts have to be designed and implemented
 - In other cases, changes to the design and implementation of existing code artifacts are needed

16.3 What is Required of Postd. Maint. Prog. (contd)?

- Adaptive and Perfective Maintenance (contd)?
 - When programs are developed
 - Specifications are produced by analysis experts
 - Designs are produced by design experts
 - Code is produced by programming experts
 - But a maintenance programmer must be expert in all three areas, and also in
 - Testing, and
 - Documentation

16.3 What is Required of Postd. Maint. Prog. (contd)?

- Conclusion
 - No form of maintenance
 - Is a task for an unsupervised beginner, or
 - Should be done by a less skilled computer professional

The Rewards of Maintenance

- Maintenance is a thankless task in every way
 - Maintainers deal with dissatisfied users
 - If the user were happy, the product would not need maintenance
 - The user's problems are often caused by the individuals who developed the product, not the maintainer
 - The code itself may be badly written
 - Postdelivery maintenance is looked down on by many software developers
 - Unless good maintenance service is provided, the client will take future development business elsewhere
 - Postdelivery maintenance is the most challenging aspect of software production — and the most thankless



The Rewards of Maintenance (contd)

- How can this situation be changed?
 - Managers must assign maintenance to their best programmers, and
 - Pay them accordingly

16.5.1 Defect Reports

- We need a mechanism for changing a product
- If the product appears to function incorrectly, the user files a defect report
 - It must include enough information to enable the maintenance programmer to recreate the problem
- Ideally, every defect should be fixed immediately
 - In practice, an immediate preliminary investigation is the best we can do

Defect Reports

- The maintenance programmer should first consult the defect report file
- It contains
 - All reported defects not yet fixed, and
 - Suggestions for working around them

If the Defect Has Been Previously Reported

- Give the information in the defect report file to the user

If it Is a New Defect

- The maintenance programmer should try to find
 - The cause,
 - A way to fix it, and
 - A way to work around the problem
- The new defect is now filed in the defect report file, together with supporting documentation

If it Is a New Defect (contd)

- The file should also contain the client's requests for perfective and adaptive maintenance
 - The contents of the file must be prioritized by the manager in charge of postdelivery maintenance
 - The next modification is the one with the highest priority
- Copies of defect reports must be circulated to all
 - Including: An estimate of when the defect can be fixed
- If the same failure occurs at another site, the user can determine
 - If it is possible to work around the defect, and
 - How long until it can be fixed

Management of Postdelivery Mainten. (contd)

- In an ideal world
 - We fix every defect immediately
 - Then we distribute the new version of the product to all the sites
- In the real world
 - We distribute defect reports to all sites
 - We do not have the staff for instant maintenance
 - It is cheaper to make a number of changes at the same time, particularly if there are multiple sites

16.5.3 Ensuring Maintainability

- Maintenance is not a one-time effort
- We must plan for maintenance over the entire life cycle
 - Design workflow — use information-hiding techniques
 - Implementation workflow — select variable names meaningful to future maintenance programmers
 - Documentation must be complete and correct, and reflect the current version of every artifact



16.5.4 The Problem of Repeated Maintenance

- The moving target problem is frustrating to the development team
- Frequent changes have an adverse effect on the maintainability of the product

The Moving Target Problem

- The problem is exacerbated during postdelivery maintenance
- The more changes there are
 - The more the product deviates from its original design
 - The more difficult further changes become
 - Documentation becomes even less reliable than usual
 - Regression testing files are not up to date
 - A total rewrite may be needed for further maintenance



The Moving Target Problem (contd)

- Apparent solution
 - Freeze the specifications once they have been signed off until delivery of the product
 - After each request for perfective maintenance, freeze the specifications for (say) 3 months or 1 year
- In practice
 - The client can order changes the next day
 - If willing to pay the price, the client can order changes on a daily basis

16.7 Postdelivery Maintenance versus Development Skills

- The skills needed for maintenance include
 - The ability to determine the cause of failure of a large product
 - The ability to function effectively without adequate documentation
 - Skills in analysis, design, implementation, and testing



16.8 Reverse Engineering

- When the only documentation for postdelivery maintenance is the code itself
 - Start with the code
 - Recreate the design
 - Recreate the specifications (extremely hard)
 - CASE tools can help (flowcharters, other visual aids)

16.8 Reverse Engineering (contd)

- What if we have only the executable code?
 - Treat the product as a black box
 - Deduce the specifications from the behavior of the current product
 - The reconstructed specifications are modified as required and a new version of the product is forward engineered from those specifications

16.13 Challenges of Postdelivery Maintenance

- The chapter describes numerous challenges
- The hardest challenge to solve
 - Maintenance is harder than development, but
 - Developers tend to look down on maintainers, and
 - Are frequently paid more